

TP Grupal: Money Laundering Analysis

Escalabilidad

Facultad de Ingeniería de la Universidad de Buenos Aires

Sistemas Distribuidos

Cátedra Roca



Integrantes:

Alumno	Padrón
Fernandez, Ignacio Agustín	111019
Molina, Taiel Alexis	109458
Serra, Luciano	110694

Introducción

Se solicita un sistema distribuido que analice el extracto de transacciones realizadas entre cuentas bancarias en busca de movimientos sospechosos que signifiquen que haya un lavado de dinero de fondo. Para ello se solicita que cada cliente pueda cargar sus datasets y a partir de los mismos recibir el resultado de las 5 consultas solicitadas.

Para la resolución serán empleados los datasets **accounts** y **transactions**. Estos serán el input del cliente y a partir de transformaciones de los mismos se obtendrán los resultados de las 5 consultas.

Se espera que el sistema permita la ejecución aislada y concurrente de distintos clientes. Esto significa que distintos nodos del sistema pueden estar trabajando con información de distintos clientes en el mismo momento. Para garantizar el aislamiento entre clientes se utilizará un *client_id* en todos los mensajes internos evitando leaks. En pos de simplificar la lectura se omitirá este *client_id* en los diagramas.

Esta condición invita a repensar los órdenes de magnitud considerados para el desarrollo del sistema, entendiendo que dados N clientes simultáneos, esos órdenes de magnitud se multiplicarán por ese valor N .

A continuación veremos una representación de las transformaciones que presentan los datos en los saltos entre instancias del sistema, así como las dependencias de secuencialidad entre las mismas.

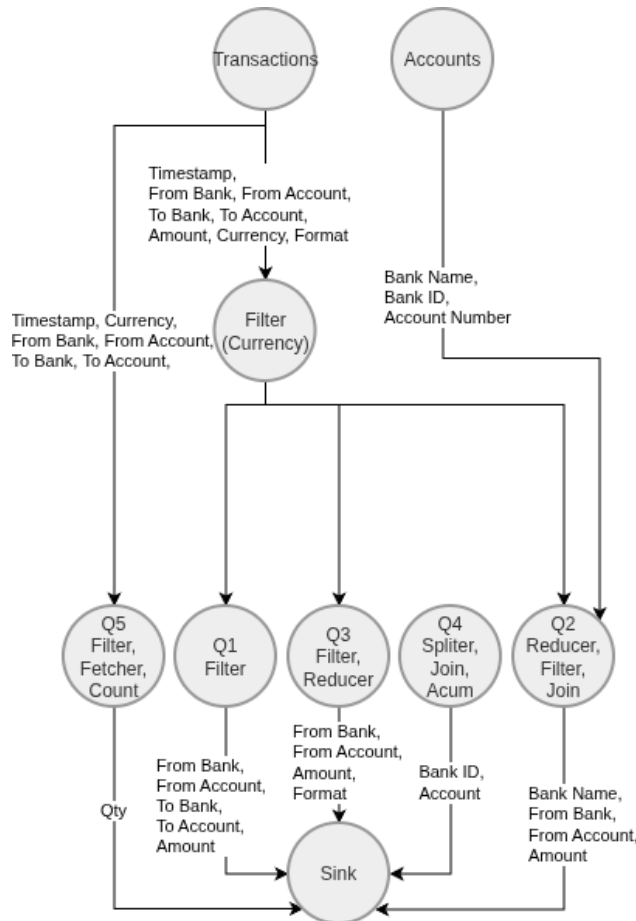


Figure 1: DAG de todo el sistema.

Sumamos detalle "haciendo zoom" en cada una de las queries entendiendo el funcionamiento aislado de cada una.

Para estos añadimos un detalle que muestra la dependencia entre cada instancia. Las relaciones "stream" implican que el nodo origen puede transmitir datos al nodo destino sin necesidad de haber procesado su correspondiente EOF.



Figure 2: DAG ejercicio 1.

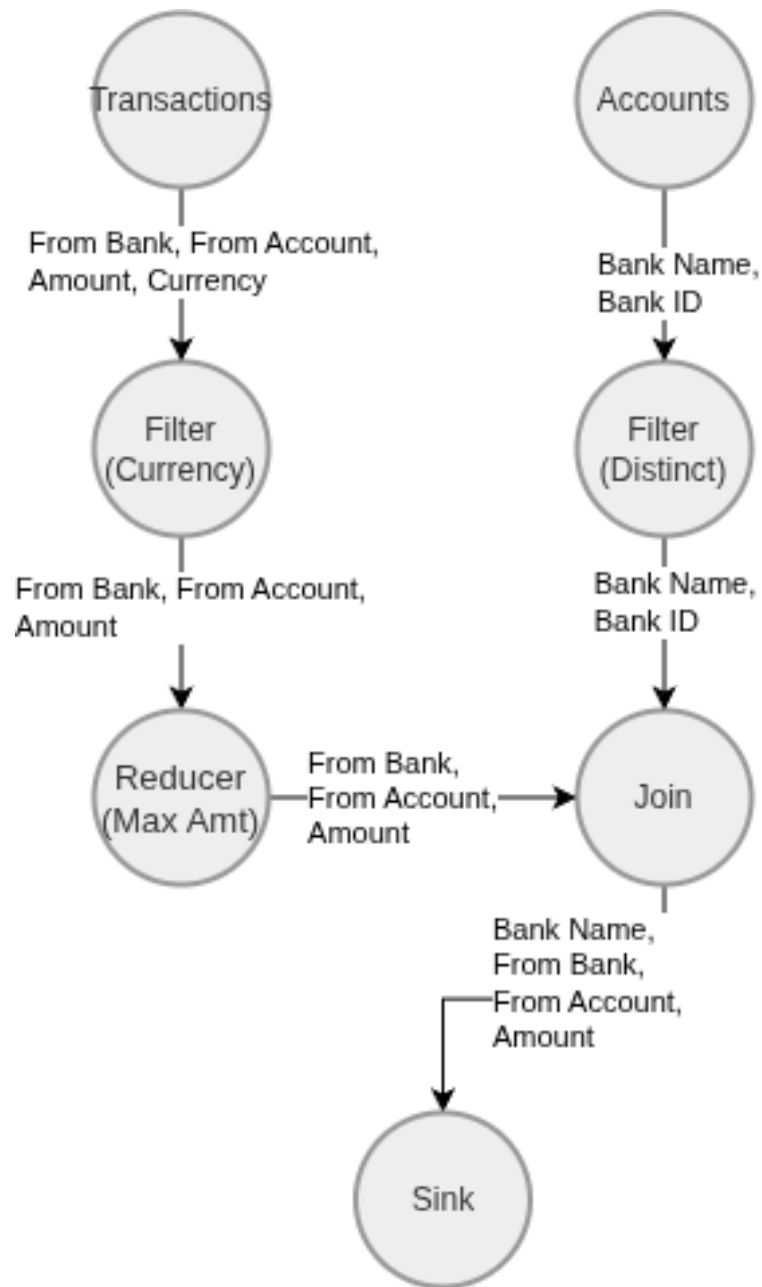


Figure 3: DAG ejercicio 2.

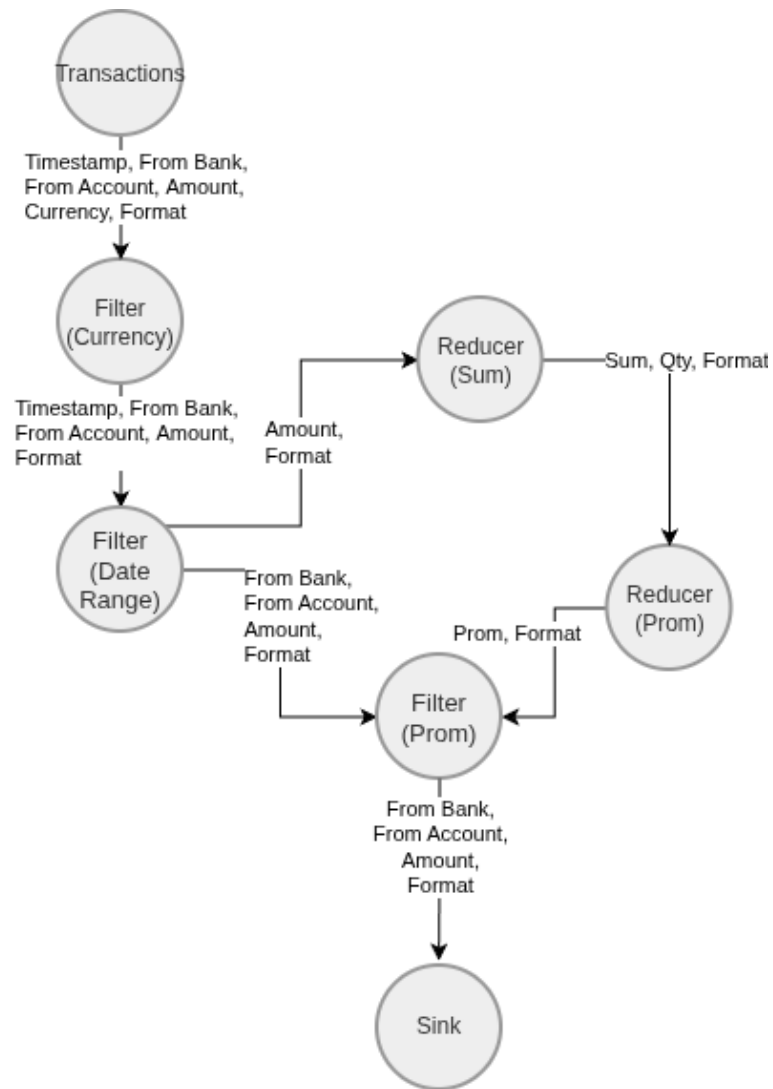


Figure 4: DAG ejercicio 3.

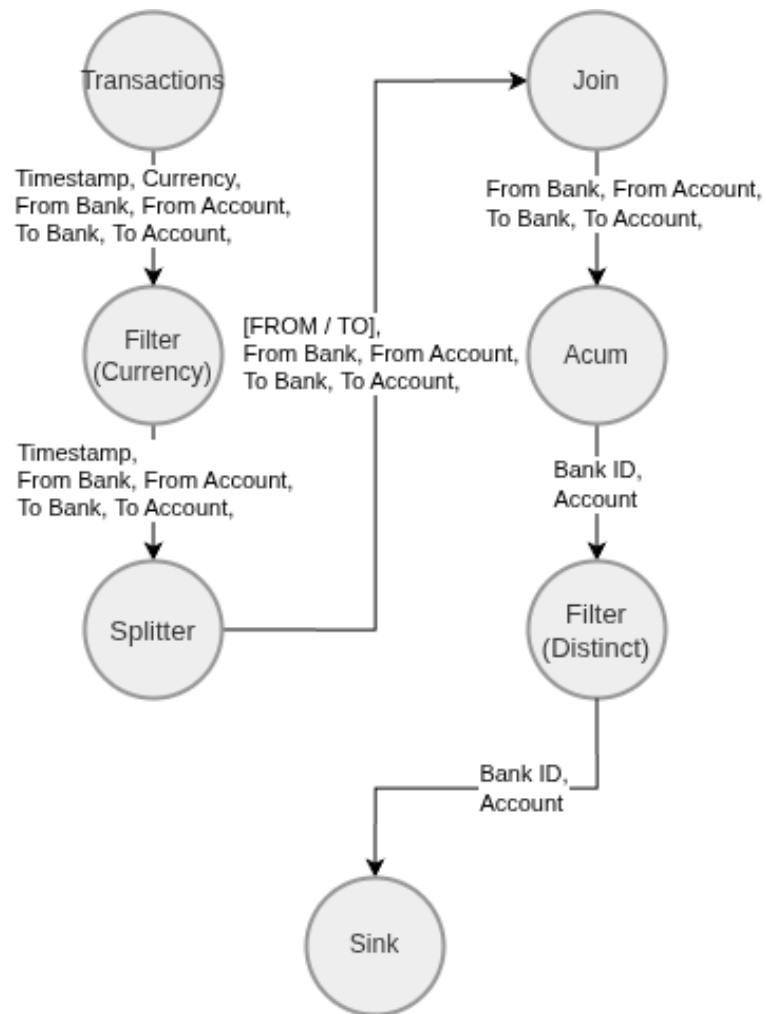


Figure 5: DAG ejercicio 4.

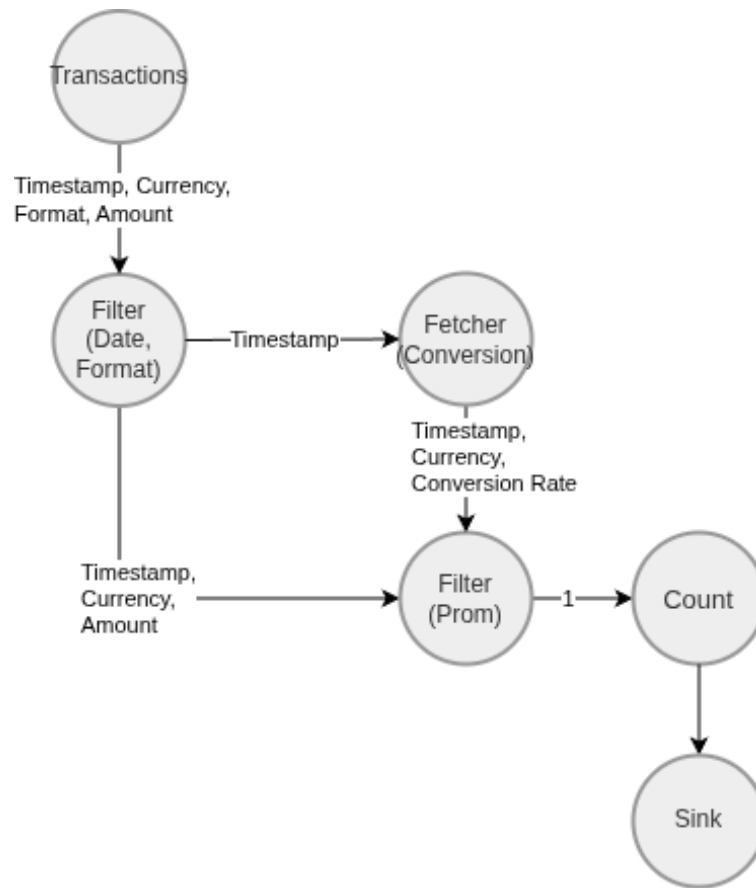


Figure 6: DAG ejercicio 5.

Posteriormente, exponemos una representación de las consultas con un diagrama de robustez para cada una.

Query I

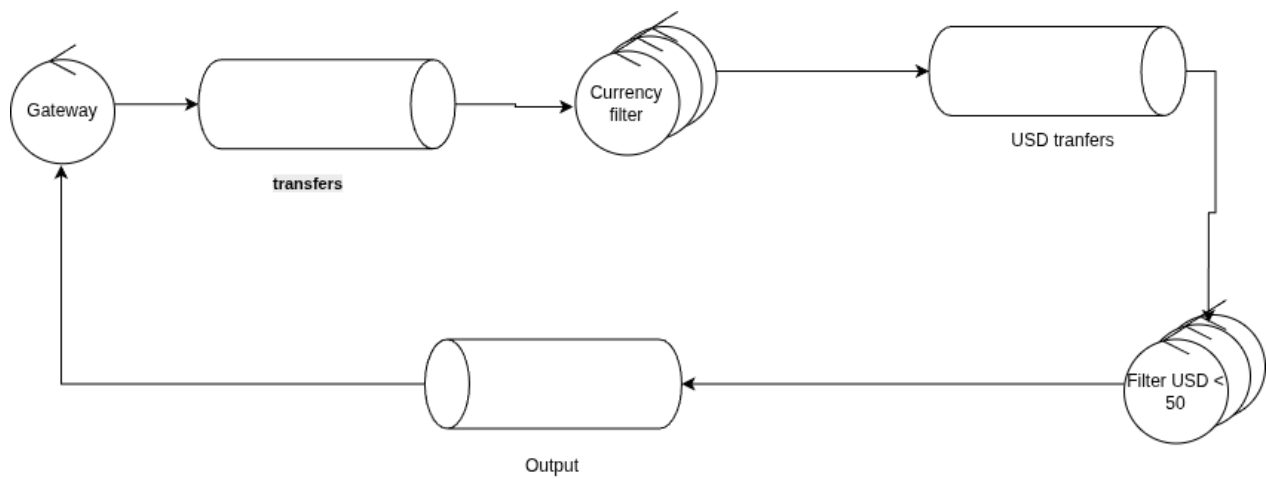


Figure 7: Query I.

Query II

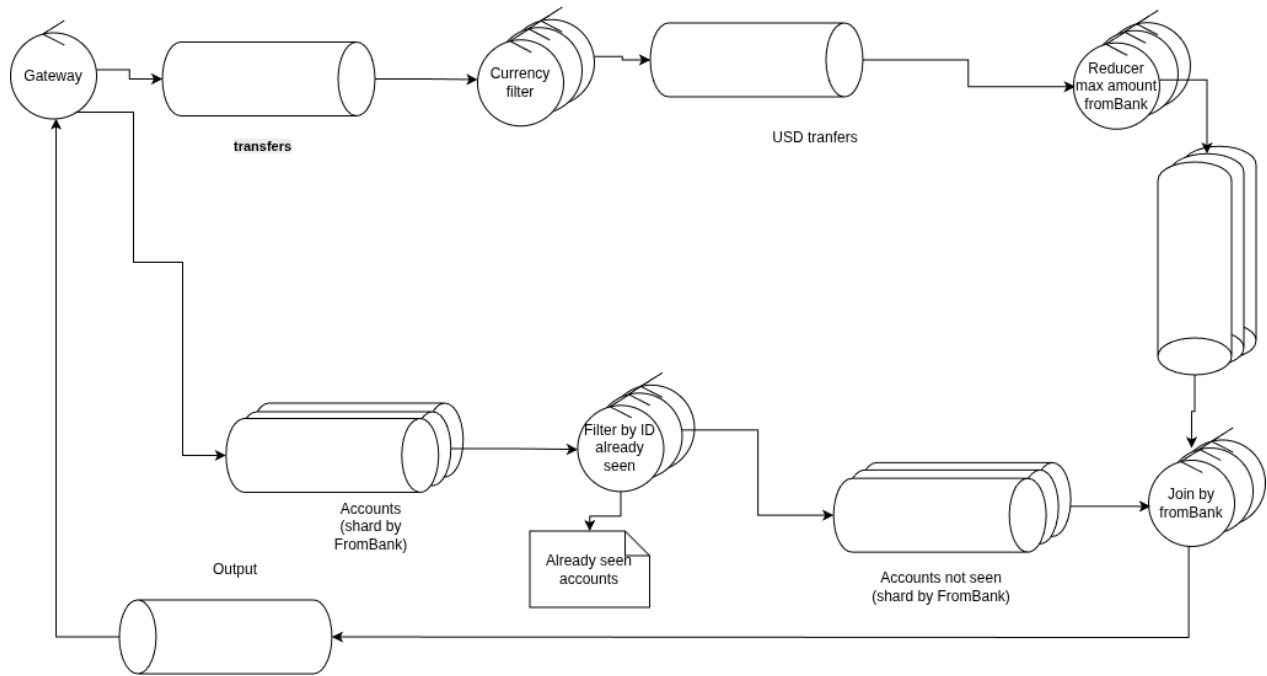


Figure 8: Query II.

Query III

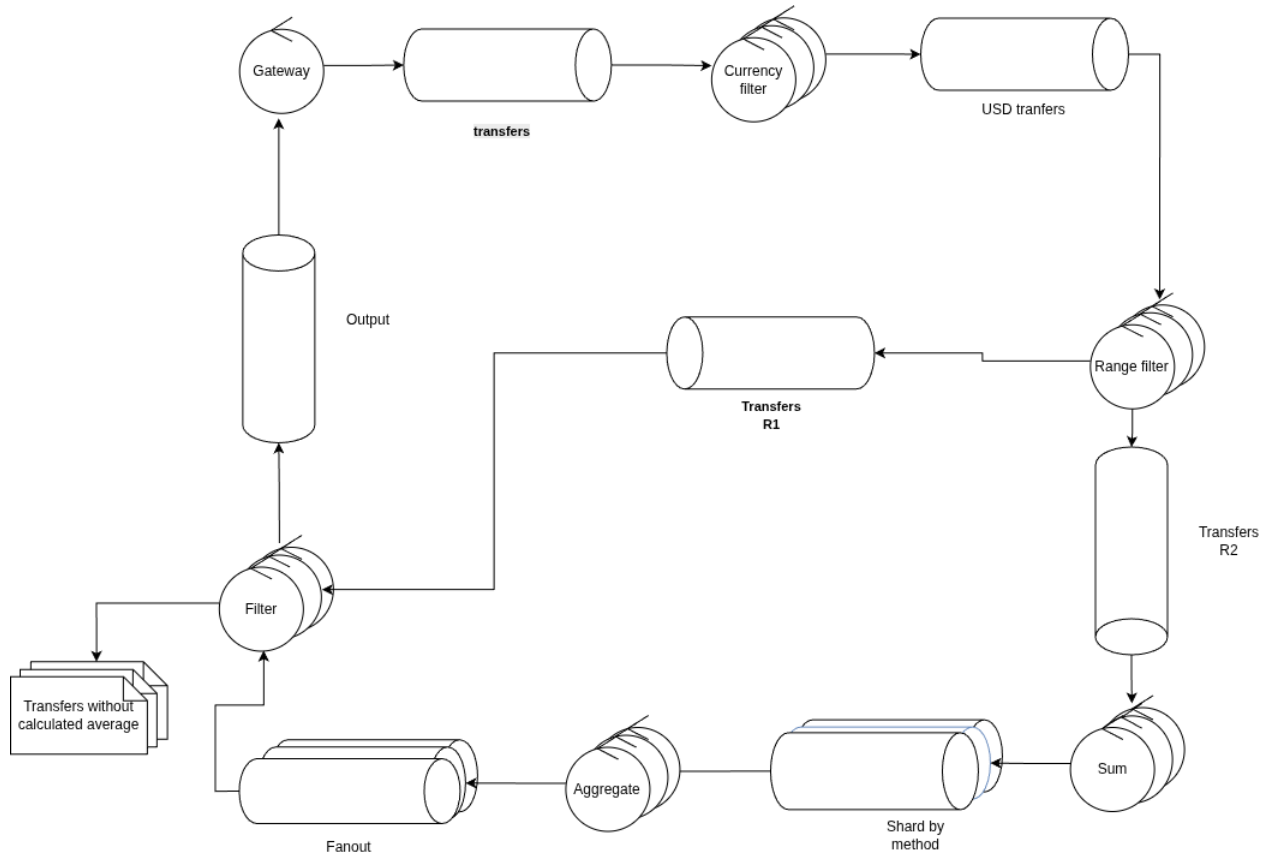


Figure 9: Query III.

Query IV

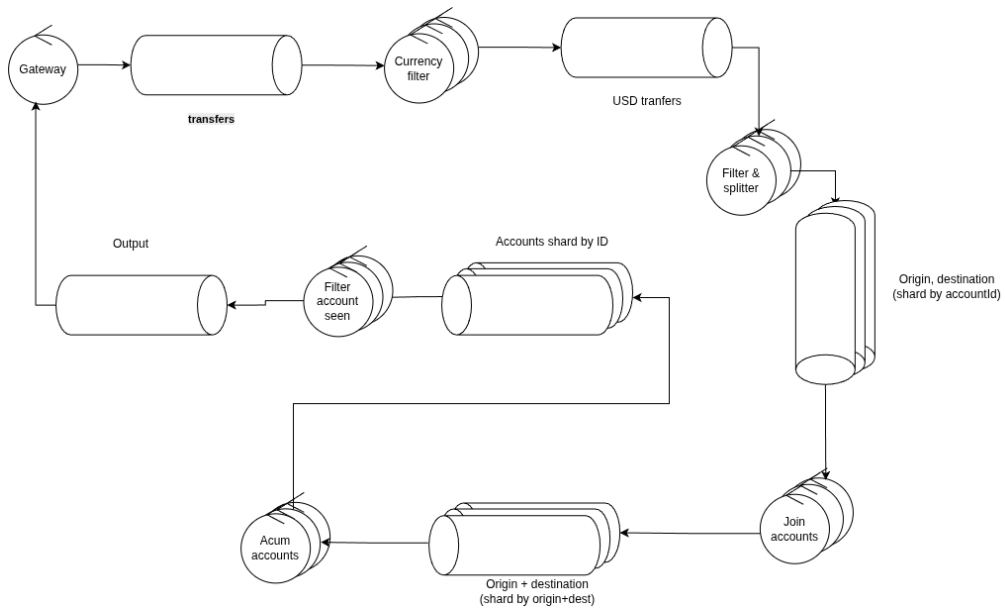


Figure 10: Query IV.

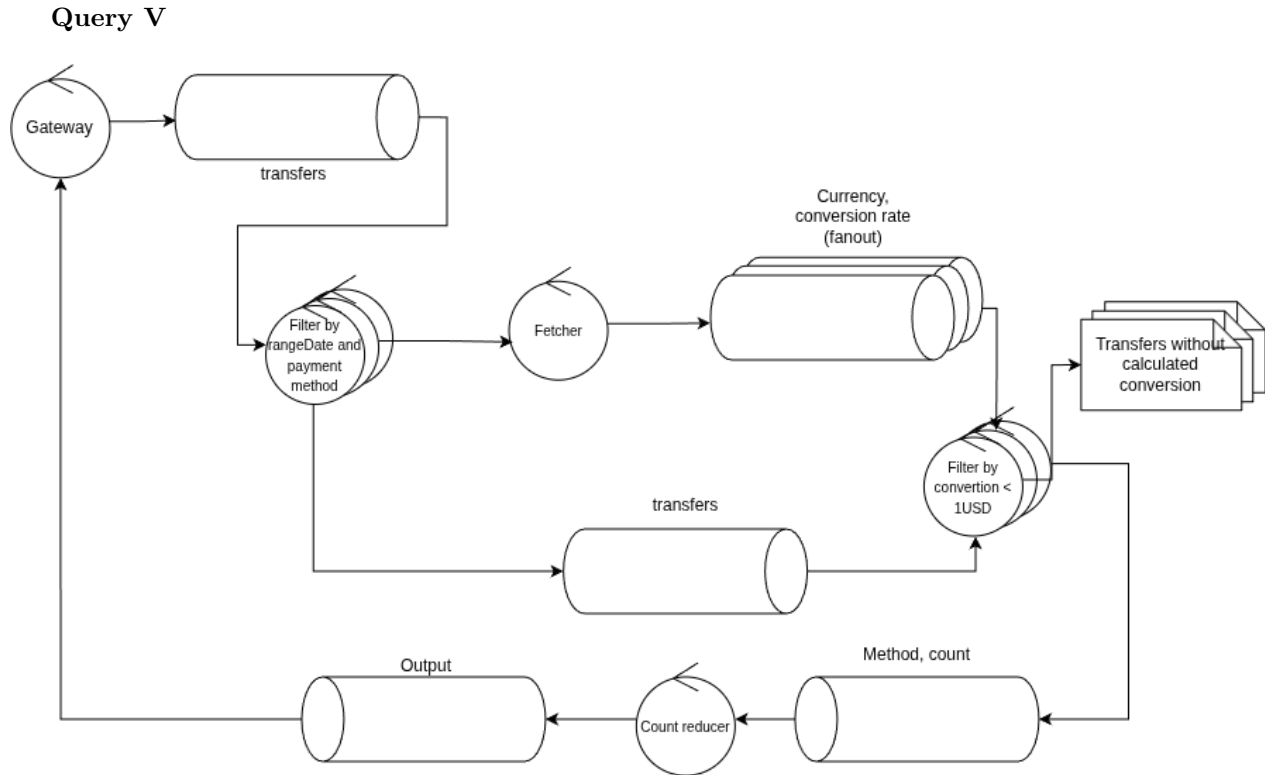


Figure 11: Query V.

Los distintos procesos que ejecutan el pipeline distribuido se pueden observar en el siguiente diagrama.

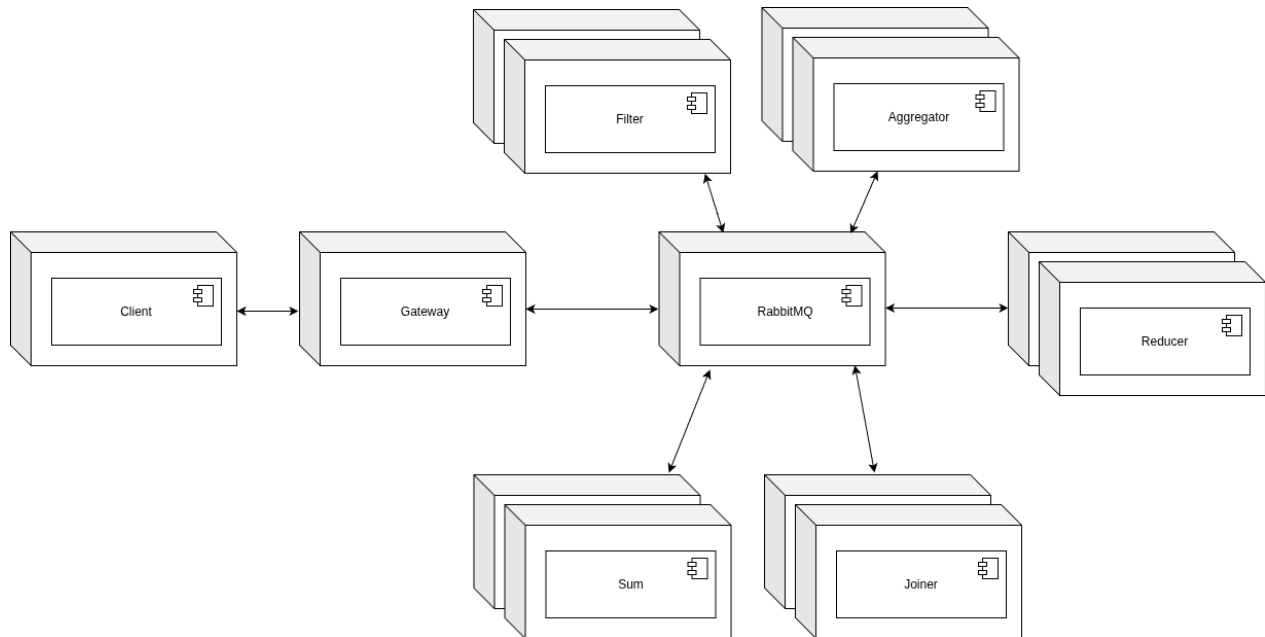


Figure 12: Diagrama de despliegue del sistema.

En él, se puede visualizar cómo se distribuyen los componentes físicos del sistema. Se puede observar que los nodos de procesamiento, como el Filter, Reducer, Joiner, Aggregator y Counter, se representan con

múltiples instancias, indicando que pueden escalar horizontalmente agregando más réplicas según la carga. Todos estos nodos se comunican a través de RabbitMQ, que centraliza el flujo de mensajes del sistema. El Client y el Gateway se despliegan como instancias únicas, siendo este último el encargado de recibir los datos del cliente y distribuirlos hacia las colas correspondientes.

A nivel paquetes de código, se presenta uno llamado **common** compartido por todos los componentes del sistema, el cual contiene los módulos de **Middleware** (wrapper de RabbitMQ), **Protocol** (serialización de mensajes) y **Models** (estructuras de datos compartidas).

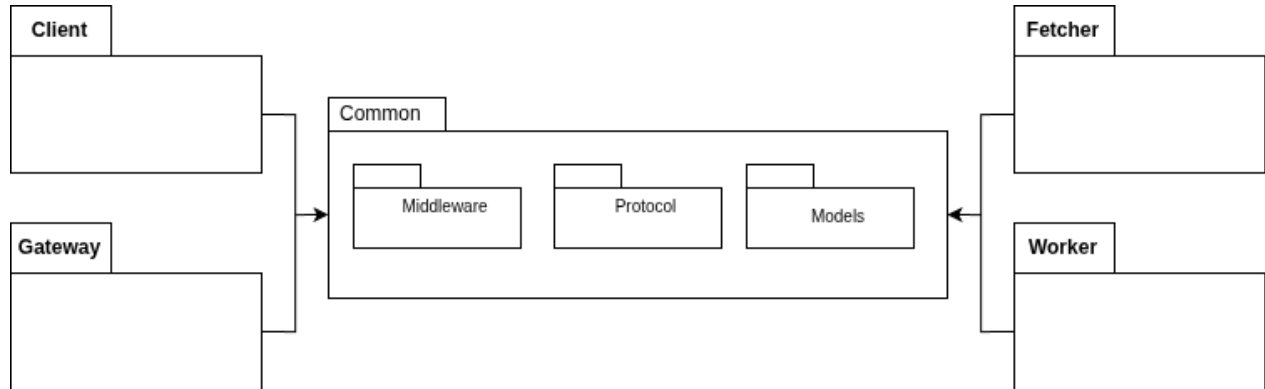
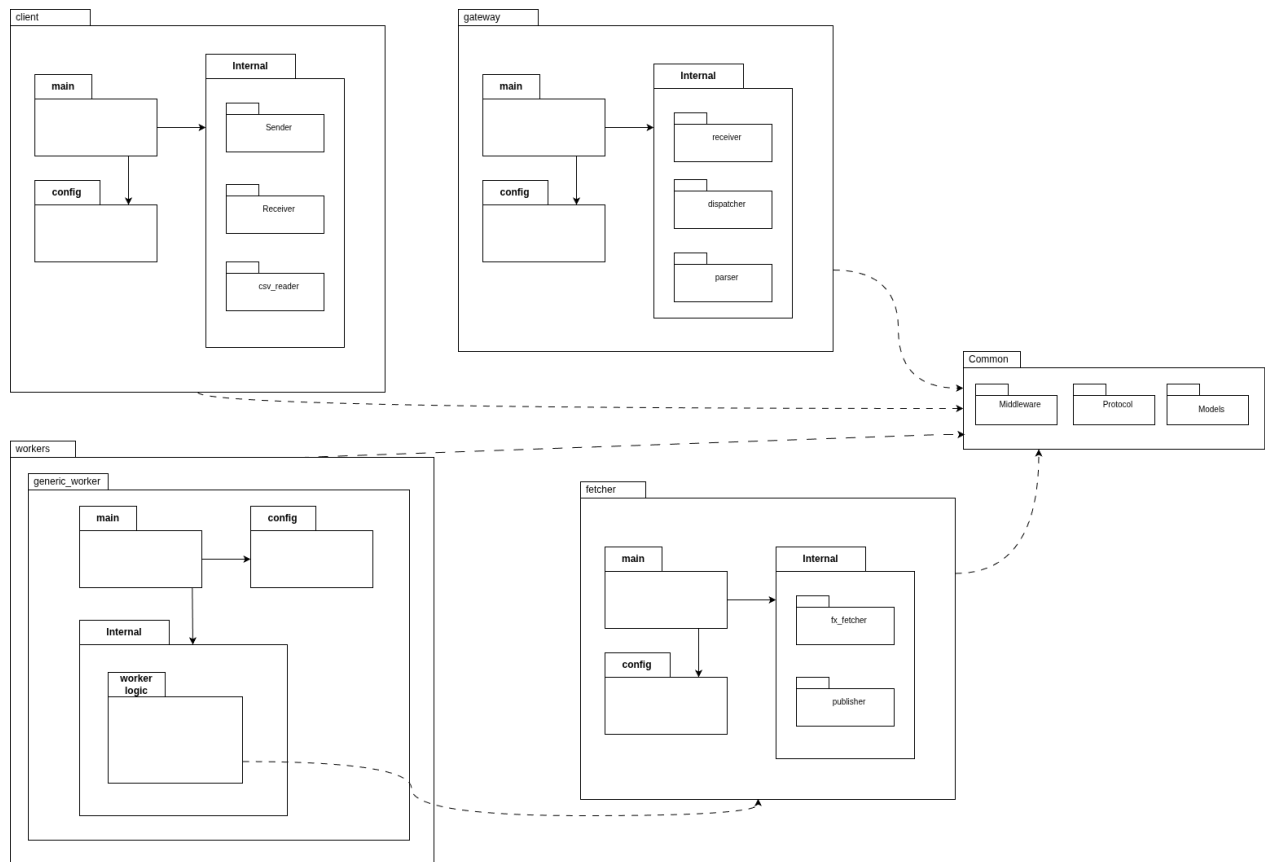


Figure 13: Vista general de dependencias entre paquetes del sistema.

Todos comparten una estructura similar: un módulo **main** como punto de entrada, un módulo **config** para las variables de entorno, y un paquete **internal** con la lógica específica de cada componente. El **Client** contiene los módulos de envío del CSV y recepción del reporte final. El **Gateway** es el punto de entrada del sistema, y se encarga de recibir los datos del cliente y distribuirlos a las colas correspondientes. El **Fetcher** consulta la API externa de cotizaciones y publica los resultados. Finalmente, el **Worker** es un componente genérico que encapsula la lógica de procesamiento, siendo instanciado con distintos comportamientos según el rol que cumpla en el pipeline (filtrado, reducción, join o agregación).



Finalmente, podemos observar el desacople entre el cliente y el procesamiento del sistema "puertas adentro". El cliente establece conexión con el gateway e inmediatamente inicia la transferencia de los archivos. Priorizamos la carga de las transferencias en primer lugar antes que las cuentas dada la cantidad de operaciones a realizar con estas, además de considerar también el volumen (y de esta forma mantener menos ocioso al sistema). El cliente envía batches de información al gateway el cual los descompone en mensajes a encolar en el exchange/queue correspondiente según la naturaleza del dato.

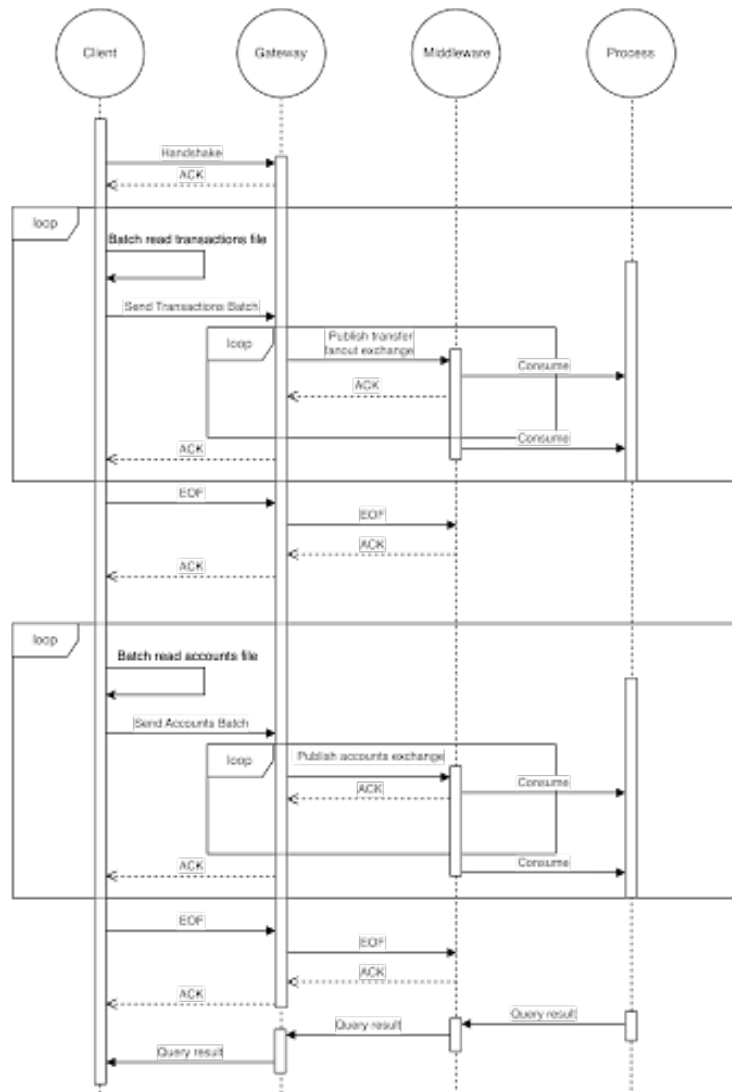


Figure 14: Diagrama de secuencia.

Tareas por hacer

Tarea	Responsable
Client + Protocol	Fernandez
Gateway	Serra
Middleware(s)	Fernandez; Molina; Serra
Algoritmo sync de EOF	Molina
Query 1	Fernandez; Molina; Serra
Query 2	Fernandez; Molina; Serra
Query 3	Serra
Query 4	Fernandez
Query 5	Molina
Tests	Molina; Serra
Docker compose	Fernandez